Cryo

Cold data finder for ONTAP NAS volumes: how it works, from the GUI to the cluster, including logic and security

1. What it is

Cryo (`cryo.py`) is a single-file Python tool with a browser GUI. It finds files on NetApp ONTAP NAS volumes (NFS and SMB data) that have not been modified or accessed for a number of days that you choose. It reads folder listings through the ONTAP REST API, so nothing is installed on the cluster and File System Analytics does not need to be enabled. It is read-only: every request it sends to ONTAP is an HTTP GET.

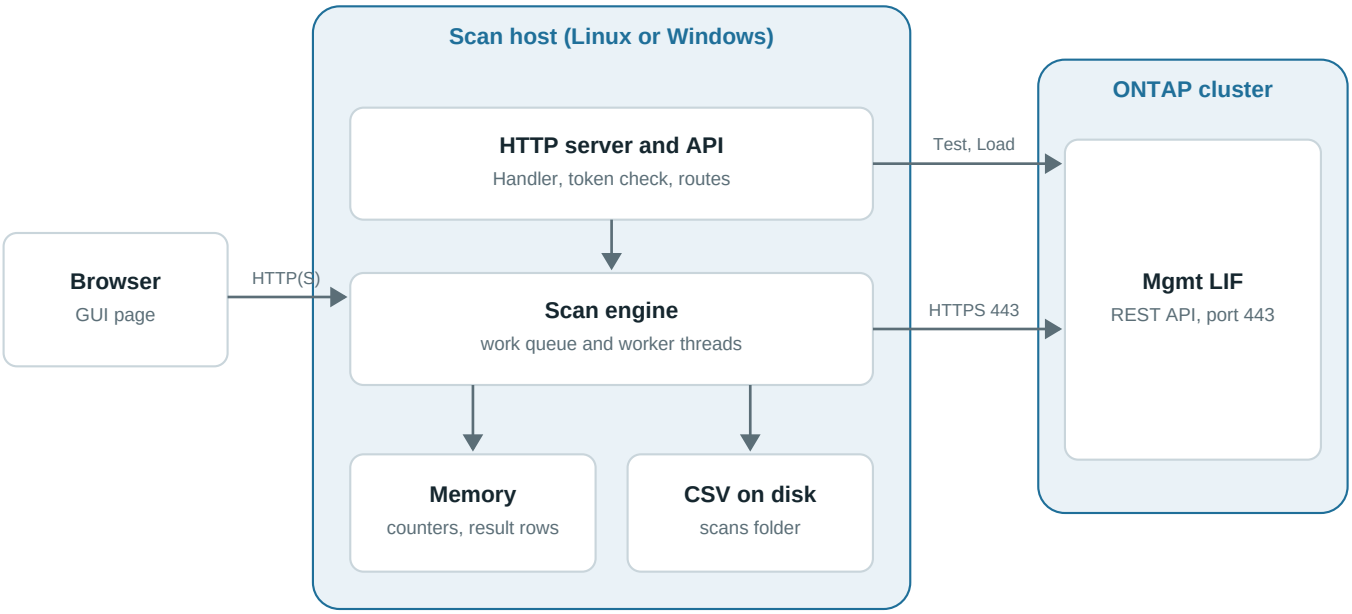
- **No extra installs.** Python 3.6 or newer, standard library only. No pip, no web framework, no internet access.
- **Air-gap friendly.** The whole GUI (HTML, CSS and JavaScript) is embedded in the script. It loads no fonts, libraries or scripts from outside.
- **Credentials stay in memory.** The ONTAP password is never written to disk.
- **Results are saved.** Each scan writes a CSV file you can download from the GUI or copy from the server.

Validation status. The tool has been tested against a mock ONTAP server and with direct tests of the exclusion, sorting and filtering logic. It has not yet been run against a real cluster. Do the first real run on a small volume and a small start folder (see section 8).

Contents

- 2. Architecture
- 3. From the GUI to the script
- 4. How it connects to ONTAP
- 5. How a scan works
- 6. Results, sorting, filters and the CSV
- 7. Security
- 8. Running it and troubleshooting
- 9. Limits and known gaps
- Appendix: settings and limits, glossary

2. Architecture



The script has four parts, all in one file.

Part	Where in the script	What it does
Embedded GUI	`PAGE` (a string of HTML, CSS and JavaScript)	The page the browser shows. Calls the script's API with fetch() and draws the results.
HTTP server	`Server`, `Handler`, `App`	Serves the page and the API. Checks the access token. Validates input. Holds the current scan.
Scan engine	`Scan`	Runs a scan in background threads. Keeps counters and result rows. Writes the CSV. Answers sort and filter requests.
ONTAP client	`Ontap`, `quote_path`, `parse_ts`	Sends the REST requests with basic authentication, TLS, retries and paging.

Threading model

- The HTTP server creates one thread per browser request, so the GUI stays responsive during a scan.
- A scan runs in its own background thread, which starts the worker threads (1 to 16, default 6).
- All scan state is shared and protected by one lock (`Scan.lock`). Workers hold it only briefly, to add a folder's results. REST calls happen outside the lock.
- Only one scan can run at a time. Starting a second returns an error until the first is stopped or finished.

3. From the GUI to the script

The GUI never talks to ONTAP. The browser talks only to the script, and the script talks to ONTAP. Every GUI action is a small JSON request to one of these routes.

Route	What it does	Calls ONTAP?
GET /	Serves the embedded page. Needs no token and contains no data.	No
GET /api/status	Counters, age buckets, largest folders, errors and the scan state. The page polls this every 1.5 seconds.	No
GET /api/results	One page of the Files table, with sort, search and filters applied.	No
GET /api/history	Lists the saved CSV files in the scans folder.	No
GET /api/file?name=	Downloads one saved CSV.	No
POST /api/test	Cluster name and ONTAP version.	Yes, 1 request
POST /api/volumes	The volume list for the picker.	Yes, 1 or a few
POST /api/scan/start	Validates the settings and starts a scan.	Yes, many (the scan)
POST /api/scan/stop	Sets the stop flag so workers exit.	No

What each button does

Button or action	Request	What the script does
Test connection	POST /api/test	Builds an ONTAP client from the form fields and calls GET /api/cluster. Shows the cluster name and version, or the error.
Load volumes	POST /api/volumes	Lists volumes (name, UUID, SVM, state, type) and fills the volume picker. The picker value is the UUID.
Start scan	POST /api/scan/start	Checks the volume is a UUID, the criteria is one of three values, and clamps the numbers. Creates a Scan and starts it.
Stop	POST /api/scan/stop	Sets a flag. Workers notice it between entries and exit. The CSV keeps what was found so far.
Progress tiles, Summary tab	GET /api/status every 1.5 s	Copies the counters under the lock and returns them. No ONTAP calls.
Files tab: sort, search, filters, paging	GET /api/results	Filters and sorts the rows already in memory and returns 100. No ONTAP calls.
Download CSV, Saved scans	GET /api/file	Sends the file from the scans folder. Only a bare file name ending in .csv is accepted.

Sorting and filtering never rescan. They only reorder or narrow the rows the scan already collected. Only **Start scan** (and Test connection and Load volumes, once each) sends requests to ONTAP. In the terminal where the script runs, you will see /api/status and /api/results lines when you use the page, and a POST /api/scan/start line only when a scan begins.

The results request

Parameter	Meaning
q	Path contains this text (not case sensitive).
sort	Up to four items like `mtime:asc,size:desc`. Keys: path, size, atime, mtime, age.
offset, limit	Paging. Limit is 1 to 1000; the GUI uses 100.
fmin, fmax	Size range in bytes (the GUI converts from KiB, MiB or GiB).
mod, acc	Not modified for at least this many days; not accessed for at least this many days.
ext	Only these file types, for example `.vmdk,.iso`.

4. How it connects to ONTAP

The script connects to the **cluster management LIF** (the cluster's management address) over HTTPS on port 443. It does not connect to each node. The cluster routes each request internally, and if the node hosting the LIF fails, ONTAP moves the LIF and the scan carries on. Always enter the cluster management address, not a node address.

The REST requests

Purpose	Request
Test connection	GET /api/cluster?fields=name,version
Volume list	GET /api/storage/volumes?fields=name,uuid,svm.name,state,type,style&max_records=500
Check the root path (once per scan)	GET /api/storage/volumes/{uuid}/files/ then, if needed, .../files/%2F, with max_records=1
List one folder (once per folder)	GET /api/storage/volumes/{uuid}/files/{path}?fields=name,type,size,bytes_used,accessed_time,modified_time&max_records=1000

Authentication and TLS

- **Authentication:** HTTP basic authentication, with the user and password typed in the GUI. It is sent only over HTTPS.
- **Certificate check:** three choices. *Verify with system CA store* (default). *Verify with CA file*, using a path on the scan host, which suits a private internal CA. *Skip verification*, which turns off all certificate and host name checks and should not be used in production.

Details that matter

- **Path encoding.** The {path} part is a single path parameter, so slashes inside it are sent as %2F (for example `projects%2Farchive_2022`). The path is relative to the volume, not an SMB share path or an NFS export path.
- **Root path.** ONTAP versions can differ in how they accept the volume root, so the script tries an empty path and then %2F and keeps whichever works.
- **Paging.** A folder with more than 1,000 entries comes back in pages. The script follows the `_links.next.href` link exactly as given until none is left.
- **Retries.** HTTP 429, 500, 502, 503 and 504, and network errors, are retried up to 4 times, waiting 1, 2, 4 and 8 seconds. Each request times out after 60 seconds. Other errors (such as 401, 403 and 404) fail that request at once.
- **Skipped entries.** The script skips `.`, `..`, `.snapshot` and `~snapshot`. Scanning snapshot folders would multiply the work and give misleading ages. Only entries of type file are checked, and only entries of type directory are walked into. Symbolic links are not followed.
- **Size.** The size used is `bytes_used` (actual space used), falling back to `size`.
- **Timestamps.** Times are parsed in a way that accepts `Z`, `+00:00` and `+0000` endings, with or without fractions of a second.

One volume at a time. The files API stays inside one volume. A junction into another volume is not followed, so scan each volume separately.

5. How a scan works

Life of a scan

1. **Validate.** The start route checks the settings and creates a Scan object that holds the settings, an ONTAP client, counters and an empty work queue. The time the scan starts is fixed, and every age is measured from it.
2. **Probe the root.** One small request confirms the credentials, the volume UUID and the root path style.
3. **Open the CSV** in the scans folder and write the header row.
4. **Seed the queue** with the start folder (blank means the volume root).
5. **Start the workers.** Each runs the loop below.
6. **Finish.** When the workers are done, the state becomes *done*, or *stopped* if you pressed Stop, or *failed* if setup failed. The CSV is closed.

The worker loop

```
while not stop_flag:
    take a folder from the queue (wait 0.5 s if empty;
        exit if nothing is queued or in progress)
    list the folder through ONTAP (follow every page)
    subfolders  -> collected to add to the queue
    files       -> excluded? counted apart. otherwise age-checked
    lock; add this folder's results to the totals; add subfolders
          to the pending count; unlock
    put the subfolders on the queue
```

Knowing when it is finished. An empty queue does not mean the scan is done, because another worker may be about to add subfolders. The scan keeps a counter of folders that are queued or being read. Workers exit only when that counter reaches zero. **Why a shared queue.** Workers take whatever folder is next, so one deep branch does not leave the others idle.

A failed folder does not stop the scan. If one folder returns an error (for example 403 permission denied), the error is recorded (shown on the Errors tab) and the scan moves on.

How a file is judged

Setting	Rule
Both modified and accessed are older (default)	The file's age is the number of whole days since the *more recent* of its two timestamps. It is cold if that age is at least the threshold.
Modified time is older	Only the modified time is used.
Accessed time is older	Only the accessed time is used.
Missing timestamp	If a timestamp needed for the chosen rule is missing, the file is not judged and is counted as having no usable timestamp.
Exclude file types	A file whose name ends with one of the listed endings (any case) is counted apart as excluded. It is not age-checked and not in the totals.

Access times can mislead. ONTAP updates access times at a coarse interval (about a day by default), and backup, antivirus and indexing tools can touch them so cold data looks warm. The default rule (both older) is the safer choice. Compare modified and accessed dates in the Files tab to see how much noise there is.

What is kept

- **Totals:** folders scanned, files checked, cold files, cold size, size of everything checked, excluded files and size.
- **Age buckets:** under 1 year, 1 to 2 years, 2 to 3 years, 3 to 5 years, over 5 years (by the file's age).
- **Largest cold folders:** cold size grouped by the first N parts of the folder path (N is the folder depth setting). The top 30 are shown.
- **Result rows:** up to 200,000 cold files in memory for the Files tab. The CSV always has every cold file.

6. Results, sorting, filters and the CSV

A result row

Position	Field	Notes
0	Path	Volume-relative, starting with /
1	Size	Bytes used
2	Accessed time	ISO date and time
3	Modified time	ISO date and time
4	Age (days)	The cold age used for buckets and the Age column
5, 6	Days since modified, days since accessed	Used by the "not modified for" and "not accessed for" filters

Sorting

The GUI offers up to three sort levels (a column and a direction for each). The script sorts the rows once per level, starting with the last level and finishing with the first. Because each sort keeps the order of rows that tie, the result is ordered by the first level, then by the second among ties, then by the third. Date columns compare as ISO text, which orders correctly. Clicking a column header makes it the first level. The quick sort buttons fill in the levels:

Quick sort	Levels
Largest files	Size, largest first
Longest unmodified	Modified date, oldest first; then size, largest first
Least recently accessed	Accessed date, oldest first; then size, largest first
Coldest, then largest	Modified oldest first; accessed oldest first; then size, largest first

Filters

Filters narrow the rows already collected. They are combined, so every filter must match. The line under the table shows how many files match and their total size. Filters only apply to the Files tab: the Summary tab and the CSV always cover the whole scan.

> **The scan only saves files that were cold at the threshold you set.** You can filter more strictly afterward (for example 730 days after scanning at 180), but you cannot loosen the threshold, the criteria or the exclusions without starting a new scan.

The CSV

One file per scan, named like `cryo_projects_20261025-143012.csv`, in the scans folder (default `./scans`). Columns: `path, bytes_used, accessed_time, modified_time, age_days`. A path that begins with =, +, - or @ gets a leading apostrophe, so spreadsheet programs do not treat it as a formula.

7. Security

What is built in

Area	What the script does
Read-only to ONTAP	Only GET requests are sent. The script cannot change, move or delete data.
Password handling	The password is sent from the browser to the script in the request body and used to build the authentication header. It is held in memory by the current scan object until the next scan replaces it or the program exits. It is never written to disk or the CSV, and the status response does not include it.
TLS to ONTAP	Certificate verification is on by default. A CA file can be given for a private CA. Skipping verification is possible but not recommended.
GUI access token	With --token (or the CRYO_TOKEN variable), every API route needs the token in an X-Token header (or a token parameter for download links). The page asks for it once per browser session. The page itself is served without a token.
GUI over TLS	--certfile and --keyfile serve the GUI over HTTPS.
Local by default	The script listens on 127.0.0.1. It warns at start-up if you bind to another address without a token or without TLS.
Input checks	The volume must be a UUID. The criteria must be one of three values. Numbers are clamped (days 1 to 36500, workers 1 to 16, depth 1 to 10). At most 50 excluded endings.
File download safety	Only a bare file name ending in .csv, inside the scans folder, can be downloaded. Folder tricks like ../ are removed.
Safe page output	File names and error text from ONTAP are escaped before being drawn, so a hostile file name cannot run script in the page.
CSV safety	Paths that start with =, +, - or @ are prefixed so spreadsheets do not run them as formulas.
Responses	Sent with Cache-Control: no-store and X-Content-Type-Options: nosniff.
No outside traffic	No CDN, fonts, updates or telemetry. The script contacts only the cluster address typed into the GUI.

Known gaps (be aware of these)

- **One shared token, no user accounts.** There is no per-user login, no audit log of who started a scan, no limit on wrong guesses, and the token is compared with ordinary equality rather than a constant-time check.
- **Token in download links.** The download links carry the token in the address, so it can appear in the script's access log and in browser history.
- **Password crosses the network.** The password goes from your browser to the script. On plain HTTP over a network, it is readable. Use TLS or an SSH tunnel.
- **With no token set, anyone who can reach the port can use the GUI** and make the scan host send requests to any address they type. There are no Origin or Host checks.
- **Results are not encrypted.** The CSV files hold file paths and ages, which are sensitive. They get the default file permissions of the account running the script.
- **The access log is printed to the terminal,** so nothing is retained unless you capture it.

Recommended production setup

1. **Least-privilege ONTAP account.** Create a REST role with read-only access to /api/cluster and /api/storage/volumes, and a service account that uses it. Limit which hosts can reach the management LIF to the scan host. Check the exact commands on your ONTAP version:

```
security login rest-role create -role cryo_ro -api /api/cluster -access readonly
security login rest-role create -role cryo_ro -api /api/storage/volumes -access readonly
security login create -user-or-group-name svc_cryo -application http \
      -authentication-method password -role cryo_ro
```

1. **Do not expose the script directly.** Bind it to 127.0.0.1 and put a reverse proxy (nginx or HAProxy) in front, with TLS from your internal CA and real user sign-in (SSO, or client certificates). Keep the token as a second layer.
2. **Always verify TLS to the cluster** with your internal CA file. Never use Skip verification.
3. **Run as a dedicated non-login user** with hardening (example below), SELinux enforcing, and a scans folder readable only by that user.
4. **Protect results.** Put the scans folder on an encrypted volume, restrict who can read it, and delete old CSVs on a schedule.
5. **Add an audit trail.** Send the script's output to syslog or your SIEM, and match it with ONTAP's own logs for the service account. A per-scan audit line (user, volume, settings, times) is a good next code change.
6. **Keep credentials off the browser.** A good next code change is to read the service account's password from a protected file on the server and remove the password box from the page.
7. **Control changes.** Keep the script in Git with review, and record the SHA-256 of the deployed file.

```
[Unit]
Description=Cryo cold data finder
After=network.target

[Service]
User=cryo
ExecStart=/usr/bin/python3 /opt/cryo/cryo.py \
    --host 127.0.0.1 --port 8080 --outdir /var/lib/cryo/scans
EnvironmentFile=/etc/cryo/token.env
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
ReadWritePaths=/var/lib/cryo
CapabilityBoundingSet=
Restart=on-failure

[Install]
WantedBy=multi-user.target

# nginx: TLS in front, proxy to the local script
server {
  listen 443 ssl;
  server_name cryo.example.local;
  ssl_certificate     /etc/pki/tls/certs/cryo.crt;
  ssl_certificate_key /etc/pki/tls/private/cryo.key;
  # add your SSO or client-certificate checks here
  location / { proxy_pass http://127.0.0.1:8080; proxy_read_timeout 120s; }
}
```

8. Running it and troubleshooting

Start it

```
python3 cryo.py                     # http://127.0.0.1:8080
python3 cryo.py --port 9000 --outdir /var/lib/cryo/scans
ssh -L 8080:127.0.0.1:8080 you@scanhost       # reach it from your workstation
```

Option	Meaning	Default
--host	Address to listen on	127.0.0.1
--port	Port to listen on	8080
--outdir	Folder for the CSV results	scans
--token	Access token for the API (or set CRYO_TOKEN)	none
--certfile, --keyfile	Serve the GUI over HTTPS	off

First real run

1. Enter the management address and a read-only account. Click **Test connection**. Then **Load volumes**.
2. Pick a small volume. Set a start folder with a few hundred files. Leave workers at 4 to 6.
3. Click **Start scan**. Watch the folders-per-second tile and the Errors tab.
4. Check a few results by hand against the files themselves, comparing the modified and accessed dates.
5. Then scan larger folders. If folders per second stops rising as you add workers, the cluster is the limit, so stop adding workers.

Choosing the number of workers

The work is waiting on the network, so threads are enough. Start at 4 to 6. More workers means more simultaneous requests on the management LIF, so watch cluster CPU while you test. Parallelism is across folders, not within one: a single folder with millions of files is read by one worker, page by page.

Troubleshooting

What you see	Likely cause and fix
HTTP 401	Wrong user or password.
HTTP 403	The account's role does not allow the API. Check the REST role.
HTTP 404 when a scan starts	Wrong volume UUID, or the start folder does not exist. The start folder is relative to the volume.
Certificate verify failed	The script does not trust the cluster's certificate. Use *Verify with CA file* and give the CA path.
Timeout or connection refused	The scan host cannot reach the management address on port 443. Check routing and firewall.
A scan is already running	Only one scan at a time. Press Stop, or wait for it to finish.
Volume must be a UUID	Use Load volumes and pick from the list, or paste the volume's UUID.
Folder errors on the Errors tab	Usually permissions or a folder removed during the scan. The scan continues and the rest of the results are still valid.
Cold counts look too low	The default rule needs both modified and accessed to be old. Try modified only, and check the excluded file types.
Lots of /api/status lines in the terminal	Normal. The page polls every 1.5 seconds.

9. Limits and known gaps

- **Speed.** One REST request per folder. A volume with tens of millions of files needs millions of requests and will be slow. Scan targeted subtrees, or consider a faster scan over an NFS mount.
- **Excluding file types does not speed up a scan.** ONTAP still returns every entry. Only skipping folders would save requests, and the tool does not have a folder-exclusion setting yet.
- **New connection per request.** Each request opens a new HTTPS connection. Reusing connections would speed up large scans.
- **Memory.** The Files tab holds up to 200,000 cold files. Beyond that the page tells you, and the CSV still has everything.
- **Stop is not instant.** Workers stop between entries, so a request in progress finishes first.
- **Results are not cached across scans.** Each Start scan walks the volume again. Changing the age threshold, the criteria or the exclusions needs a new scan. Sorting and filtering do not.
- **Access times are coarse and can be touched by other tools** (see section 5).
- **Counts are as of the scan start.** Files changed during a long scan may be judged against slightly old information.
- **Not yet tested on a real cluster.** Tested with a mock server and direct logic tests only.

Appendix A. Settings and limits

Item	Value
Python	3.6 or newer, standard library only
Workers	1 to 16 (default 6)
Age threshold	1 to 36,500 days (default 180); a file is cold at that many whole days or more
Folder depth for grouping	1 to 10 (default 2)
Excluded file endings	Up to 50
Entries per ONTAP page	1,000
Request timeout and retries	60 seconds; up to 4 retries with 1, 2, 4, 8 second waits
Result rows in memory	200,000 (the CSV is not limited)
Folder errors kept	500 (the GUI shows the latest 100; the total count is always shown)
Sort levels in a request	Up to 4 (the GUI offers 3)
Rows per page in the Files tab	100 (the API allows up to 1,000)

Appendix B. Glossary

Term	Meaning
LIF	Logical interface: a network address on the cluster. The management LIF is the address the REST API answers on.
SVM	Storage virtual machine: the unit that serves a set of volumes to clients.
Volume UUID	The unique ID of a volume, used in every files request.
Junction	The place where one volume is mounted inside another volume's namespace.
FSA	File System Analytics, an ONTAP feature that indexes files in the background. This tool does not need it.
atime, mtime	Last accessed time and last modified time of a file.
Cold data	Files not modified or accessed for at least the threshold you chose.
Basic authentication	Sending the user and password in an HTTP header. Safe only over TLS.

Appendix C. Charts tab

The Charts tab draws five charts from the results of the current scan. The charts are plain SVG drawn by the page's own JavaScript, so they need no chart library and work in an air-gapped environment. Like the table, they read results already collected on the scan host: they send nothing to ONTAP and need no rescan.

Chart	What it shows	Data it uses
Cold versus the rest	Cold size as a share of all scanned size, as a ring with the percentage in the middle.	cold_bytes and total_bytes from the status response
Cold data by age	Cold size in each age bucket (under 1 year up to over 5 years), as vertical bars.	The age buckets
Where the cold data is	A treemap: each box is a folder, and its area is its cold size. The largest 30 folders are shown, plus one grey box for everything else. Clicking a box opens the Files tab filtered to that folder.	The largest cold folders, grouped at the folder depth setting
By file type	The ten biggest file types by cold size, plus an Other bar, with file counts. Clicking a bar opens the Files tab filtered to that type.	A new per-extension total kept during the scan
By year last modified	Cold size by the year each file was last modified, with an unknown group for files without a usable time.	A new per-year total kept during the scan

How the file type is decided

The type is the ending of the file name after the last dot, in lower case. The endings .tar.gz, .tar.bz2 and .tar.xz are treated as one type. A name with no dot, or only a leading dot such as .bashrc, is counted as (none).

What the charts cover

- The charts cover the whole scan. They do not follow the filters on the Files tab.
- Files excluded by type are not in any chart, in line with the rest of the results.
- The folder treemap uses the 30 largest cold folders at the chosen depth. Raising the depth setting before a scan gives a finer map. Boxes that are too small for a label show the folder name and size when you point at them.
- Charts update while a scan runs, so early charts show partial results.

Two small additions to the scan. To draw the type and year charts, the scan now keeps a running total per file type and per modified year, in memory. They are included in the status response as `types` (top 12) and `years`. This adds a few kilobytes of memory and does not add any ONTAP requests.